

Binární vyhledávací strom

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

03.06.2026

CC BY-NC-SA 4.0

Binární vyhledávací strom

Binární vyhledávací strom

Binární vyhledávací strom

O **binárním vyhledávacím stromu** (*Binary Search Tree*, BST) mluvíme, když binární strom splňuje jedno přidání pravidlo:

Binární vyhledávací strom

O **binárním vyhledávacím stromu** (*Binary Search Tree*, BST) mluvíme, když binární strom splňuje jedno přidané pravidlo:

Pro **každý** uzel platí:

Binární vyhledávací strom

O **binárním vyhledávacím stromu** (*Binary Search Tree*, BST) mluvíme, když binární strom splňuje jedno přidané pravidlo:

Pro **každý** uzel platí:

- Všechny hodnoty v **levém** podstromu jsou **menší**

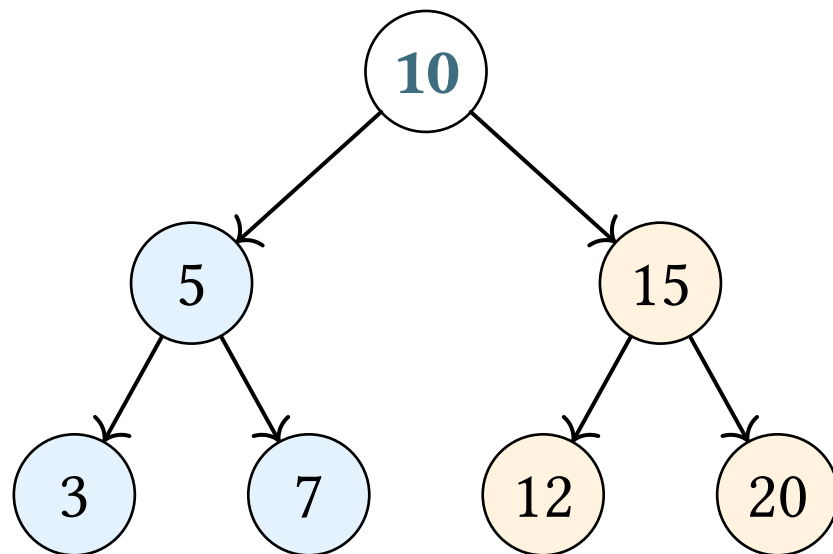
Binární vyhledávací strom

O **binárním vyhledávacím stromu** (*Binary Search Tree*, BST) mluvíme, když binární strom splňuje jedno přidání pravidlo:

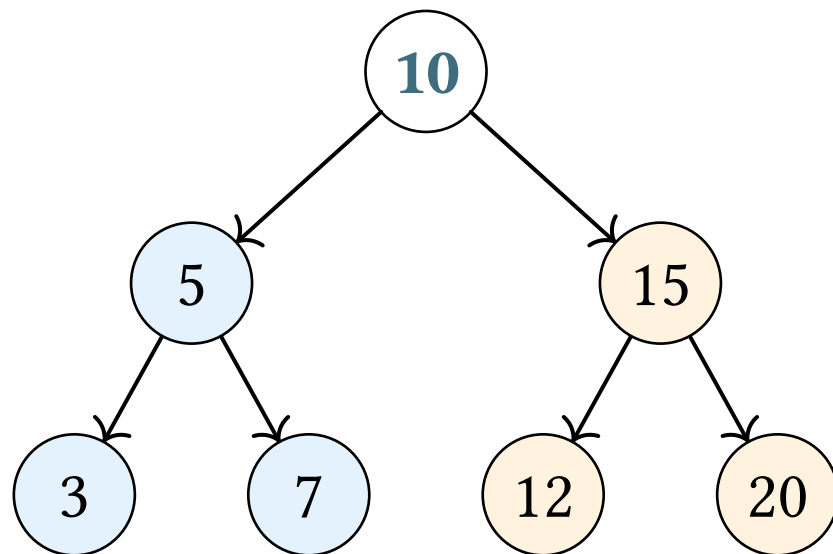
Pro **každý** uzel platí:

- Všechny hodnoty v **levém** podstromu jsou **menší**
- Všechny hodnoty v **pravém** podstromu jsou **větší**

Binární vyhledávací strom



Binární vyhledávací strom



Modré < 10

Oranžové > 10

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání		

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení		

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Smazání		

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Smazání	$\mathcal{O}(n)$	

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Smazání	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$

Motivace pro BST

Vzpomeňte si na **binární vyhledávání** v poli, tam jsme také porovnávali s prostředním prvkem a šli doleva nebo doprava

Operace	Seřazené pole	BST (vyvážený)
Vyhledávání	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Smazání	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$

BST umožňuje **efektivně vkládat i mazat**, na rozdíl od pole

Operace na BST

Vyhledávání

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** →

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** →

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** → jdi **doprava**

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** → jdi **doprava**
4. Je-li **rovna** →

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** → jdi **doprava**
4. Je-li **rovna** → **nalezeno**

Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** → jdi **doprava**
4. Je-li **rovna** → **nalezeno**
5. Je-li uzel NULL →

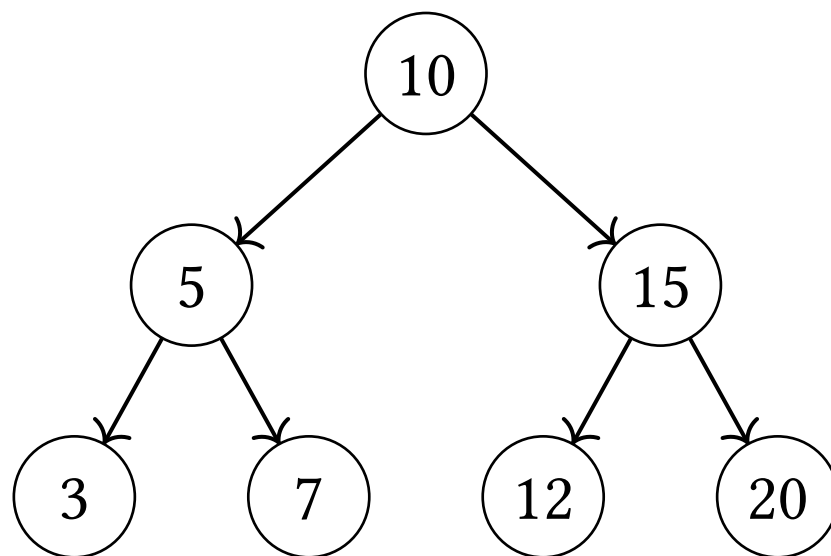
Vyhledávání

Algoritmus:

1. Porovnej hledanou hodnotu s aktuálním uzlem
2. Je-li **menší** → jdi **doleva**
3. Je-li **větší** → jdi **doprava**
4. Je-li **rovna** → **nalezeno**
5. Je-li uzel NULL → **nenalezeno**

Vyhledávání

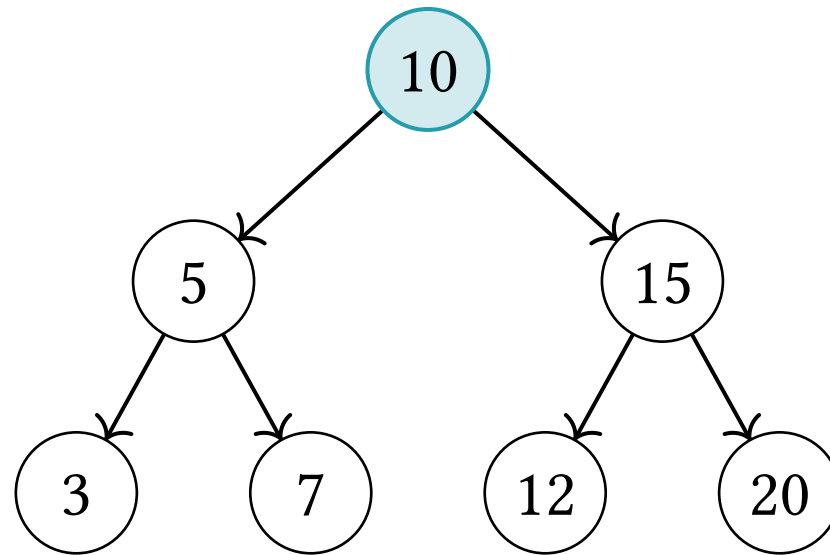
Hledáme hodnotu 12:



Začínáme u kořene

Vyhledávání

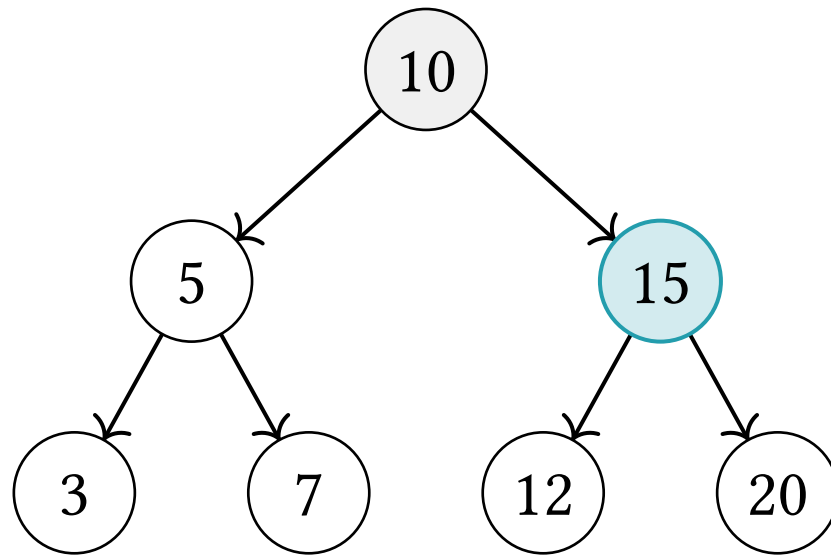
Hledáme hodnotu 12:



$12 > 10 \rightarrow$ jdeme **doprava**

Vyhledávání

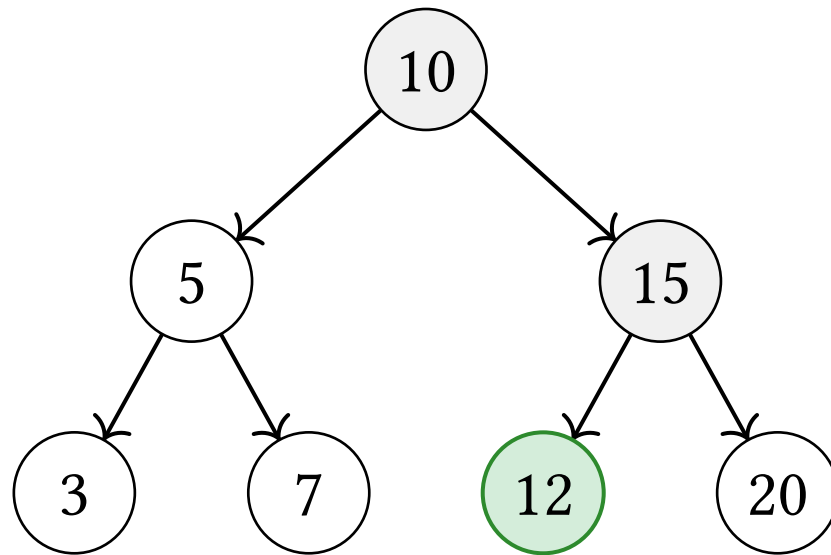
Hledáme hodnotu 12:



$12 < 15 \rightarrow$ jdeme **doleva**

Vyhledávání

Hledáme hodnotu 12:



$12 == 12 \rightarrow$ **nalezeno!**

Vyhledávání

Jakou složitost má vyhledávání v BST?

Vyhledávání

Jakou složitost má vyhledávání v BST?

Složitost: $\mathcal{O}(h)$, kde h je výška stromu

Vyhledávání

Jakou složitost má vyhledávání v BST?

Složitost: $\mathcal{O}(h)$, kde h je výška stromu

- Vyvážený strom: $h = \log_2 n \rightarrow$

Vyhledávání

Jakou složitost má vyhledávání v BST?

Složitost: $\mathcal{O}(h)$, kde h je výška stromu

- Vyvážený strom: $h = \log_2 n \rightarrow \mathcal{O}(\log n)$

Vyhledávání

Jakou složitost má vyhledávání v BST?

Složitost: $\mathcal{O}(h)$, kde h je výška stromu

- Vyvážený strom: $h = \log_2 n \rightarrow \mathcal{O}(\log n)$
- Degenerovaný strom: $h = n \rightarrow$

Vyhledávání

Jakou složitost má vyhledávání v BST?

Složitost: $\mathcal{O}(h)$, kde h je výška stromu

- Vyvážený strom: $h = \log_2 n \rightarrow \mathcal{O}(\log n)$
- Degenerovaný strom: $h = n \rightarrow \mathcal{O}(n)$

Vkládání

Vkládání

Vkládání funguje **stejně jako vyhledávání**, hledáme správné místo pro nový uzel

Vkládání

Vkládání funguje **stejně jako vyhledávání**, hledáme správné místo pro nový uzel

Algoritmus:

1. Projdi strom jako při vyhledávání

Vkládání

Vkládání funguje **stejně jako vyhledávání**, hledáme správné místo pro nový uzel

Algoritmus:

1. Projdi strom jako při vyhledávání
2. Když narazíš na NULL →

Vkládání

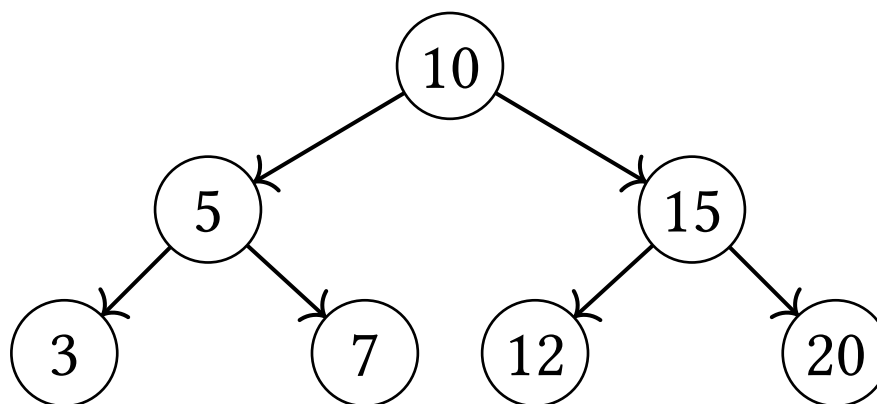
Vkládání funguje **stejně jako vyhledávání**, hledáme správné místo pro nový uzel

Algoritmus:

1. Projdi strom jako při vyhledávání
2. Když narazíš na NULL → vlož nový uzel na toto místo

Vkládání

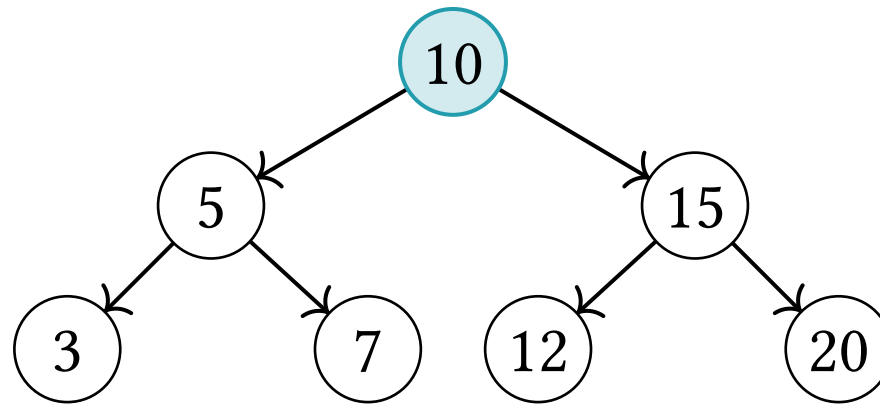
Vkládáme hodnotu 13:



Začínáme u kořene

Vkládání

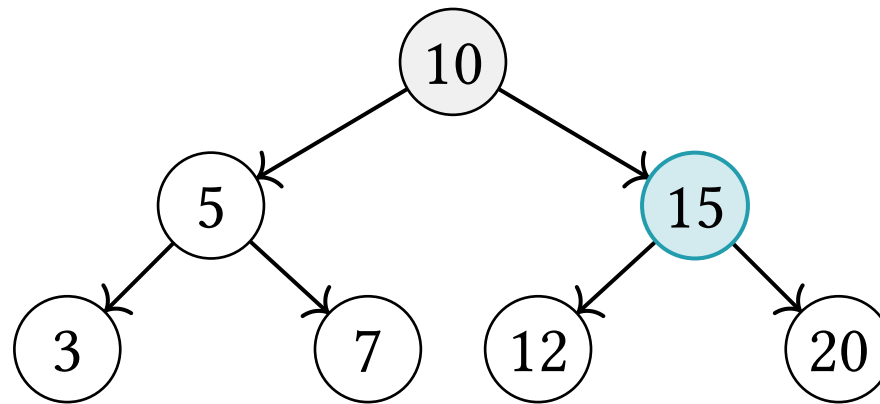
Vkládáme hodnotu 13:



$13 > 10 \rightarrow$ jdeme **doprava**

Vkládání

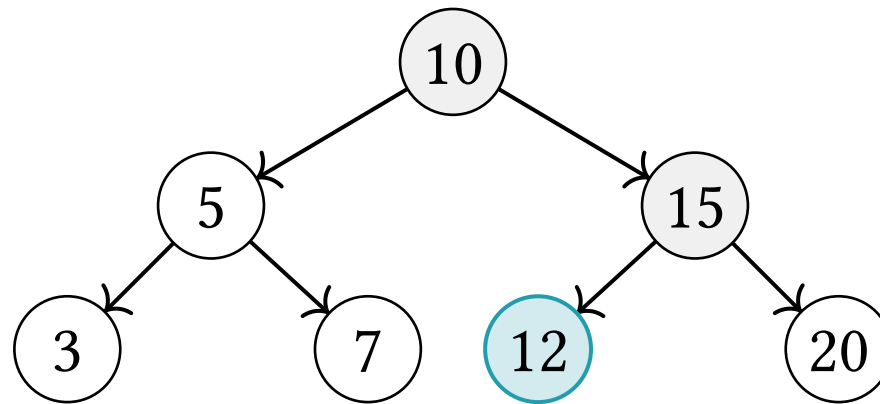
Vkládáme hodnotu 13:



$13 < 15 \rightarrow$ jdeme **doleva**

Vkládání

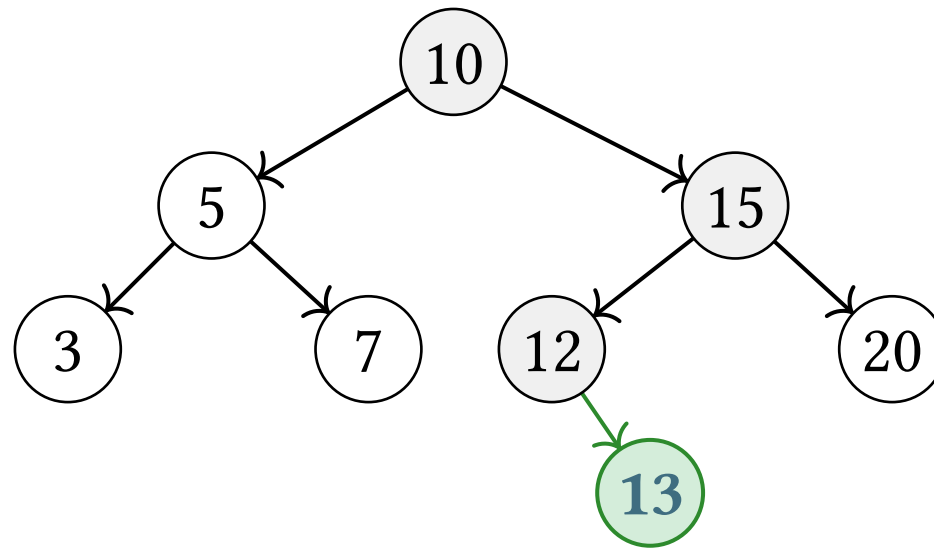
Vkládáme hodnotu 13:



$13 > 12 \rightarrow$ jdeme **doprava** $\rightarrow$ NULL

Vkládání

Vkládáme hodnotu 13:



Vložíme **13** jako pravého potomka uzlu 12

Vkládání

Jakou složitost má vkládání do BST?

Vkládání

Jakou složitost má vkládání do BST?

Také $\mathcal{O}(h)$, protože vkládání prochází stromem stejně jako vyhledávání

Degenerovaný strom

Degenerovaný strom

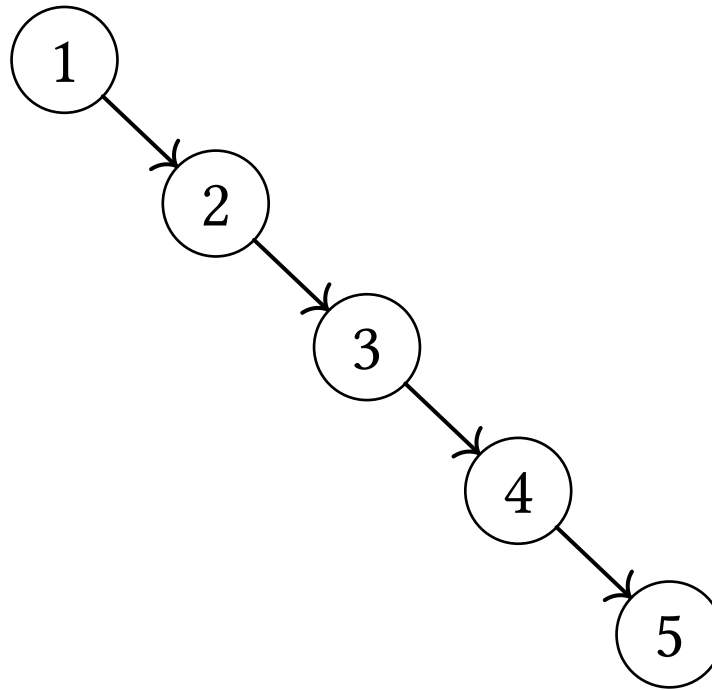
Co se stane, když vložíme prvky **v seřazeném pořadí**?

Degenerovaný strom

Co se stane, když vložíme prvky **v seřazeném pořadí**?

Vložíme postupně: 1, 2, 3, 4, 5

Degenerovaný strom



Degenerovaný strom

Strom degeneroval na **spojový seznam** a vyhledávání je

Degenerovaný strom

Strom degeneroval na **spojový seznam** a vyhledávání je $\mathcal{O}(n)$

Degenerovaný strom

Strom degeneroval na **spojový seznam** a vyhledávání je $\mathcal{O}(n)$

Řešením jsou **samovyvažovací stromy** (AVL, Red-Black)

AVL si ukážeme na příští přednášce

Shrnutí

Shrnutí

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání i vkládání mají složitost $\mathcal{O}(h)$, kde h je výška stromu

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání i vkládání mají složitost $\mathcal{O}(h)$, kde h je výška stromu
- Pro vyvážený strom: $\mathcal{O}(\log n)$

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání i vkládání mají složitost $\mathcal{O}(h)$, kde h je výška stromu
- Pro vyvážený strom: $\mathcal{O}(\log n)$
- Při vkládání seřazených dat strom degeneruje na spojový seznam $\rightarrow \mathcal{O}(n)$

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání i vkládání mají složitost $\mathcal{O}(h)$, kde h je výška stromu
- Pro vyvážený strom: $\mathcal{O}(\log n)$
- Při vkládání seřazených dat strom degeneruje na spojový seznam $\rightarrow \mathcal{O}(n)$
- BST je **dynamická** varianta binárního vyhledávání v seřazeném poli

Shrnutí

- V **BST** platí: levý podstrom < uzel < pravý podstrom
- Vyhledávání i vkládání mají složitost $\mathcal{O}(h)$, kde h je výška stromu
- Pro vyvážený strom: $\mathcal{O}(\log n)$
- Při vkládání seřazených dat strom degeneruje na spojový seznam $\rightarrow \mathcal{O}(n)$
- BST je **dynamická** varianta binárního vyhledávání v seřazeném poli
- Řešením degenerace jsou samovyvažovací stromy (AVL, Red-Black)